

Documentación del Agente de IA

Tribunal de Arquitectura de Bases de Datos — Auditoría y rediseño de GathelDB

Proyecto: Caso #3 | Base de datos: GathelDB | Motor: SQL Server 2022

34/100

Evaluación inicial



88/100

Evaluación final

Resumen ejecutivo

Se diseñó un agente de IA especializado para auditar el modelo de datos de Gathel antes de su implementación. En la primera revisión, el diseño obtuvo una calificación de 34/100 por problemas de normalización, duplicación de saldos y ausencia de trazabilidad. Tras aplicar las correcciones señaladas por el agente —catálogos normalizados, patrón Ledger, unificación de monedas y un esquema de auditoría real—, el diseño alcanzó 88/100 en la segunda evaluación, quedando listo para producción con un conjunto acotado de recomendaciones finales.

Ficha del agente

Nombre del agente	Tribunal de Arquitectura de Bases de Datos
Rol asignado	Ingeniero Principal de Datos y Arquitecto Senior de Bases de Datos con más de 20 años de experiencia, con instrucción de calificar numéricamente, señalar errores críticos, explicar sus consecuencias y proponer soluciones bajo estándares empresariales.
Objetivo	Auditar técnicamente el diseño de la base de datos del proyecto Gathel para identificar debilidades de normalización, riesgos de escalabilidad, vulnerabilidades de seguridad y malas prácticas de modelado antes de la implementación.

Aspectos evaluados

- Normalización
- Economía del juego
- Integración con inteligencia artificial
- Escalabilidad
- Seguridad
- Auditoría y trazabilidad
- Rendimiento

Primera evaluación — 34/100

El diseño inicial presentó debilidades estructurales que comprometían la integridad y escalabilidad del sistema:

- Uso excesivo de nvarchar para representar estados y tipos, sin control de valores válidos.
- Duplicación de saldos (puntos y dinero) directamente en la tabla de jugadores, sin historial.
- Ausencia del patrón Ledger para sustentar movimientos de dinero y puntos.
- Manejo incorrecto de múltiples monedas, sin un modelo unificado.
- Configuraciones de negocio estáticas, sin vigencia temporal ni historial.
- Uso ineficiente de hashes y tokens de sesión (tipos de dato sobredimensionados).

Mejoras implementadas: antes y después

Cada hallazgo del agente se tradujo en un cambio concreto al modelo de datos. La siguiente tabla resume las correcciones más relevantes aplicadas entre la primera y la segunda evaluación:

Aspecto	Antes (34/100)	Después (88/100)
Saldos del jugador	pointsBalance y moneyBalance duplicados directamente en Players, sin historial.	Modelo Wallets unificado: una billetera por jugador/moneda, alimentada por Transactions (patrón Ledger).
Estados y tipos	Campos nvarchar libres (status, type, outcome) sin control de valores válidos.	Catálogos normalizados: TransactionTypes, PropositionStatuses, PlayerStatuses, AIJobStatuses, etc.
Retiros de dinero	Tabla Withdrawals separada, redundante con la lógica de transacciones.	Eliminada: el retiro es un TransactionType más, con detalle en TransactionDetails.
Contraseñas	password en nvarchar(255) sin algoritmo de hash especificado.	varbinary(64) para el hash real, con el salt en una columna independiente.
Sesiones	sessionToken en nvarchar(500), formato no estandarizado.	char(64): hash SHA-256 en hexadecimal.
Configuración de negocio	PointsConfig como registro estático único, sin historial de cambios.	PlatformConfig con effectiveFrom / effectiveTo para vigencia temporal.
Auditoría	Sin una tabla real de auditoría de operaciones.	AuditLog con entityName, operation, performedBy, oldSnapshot y newSnapshot en JSON.
Integración con IA	AIAnalysisJobs mezclaba la solicitud y la respuesta del modelo en una sola tabla.	Separado en AIProviders, AIModels, AIAnalysisJobs (solicitud) y AIAnalysisResults (respuesta).
Penalizaciones	Penalties sin historial de configuración ni trazabilidad de aplicación.	PenaltyCatalog (vigencia temporal) + PenaltyTransactions (registro de cada aplicación).
Monedas y puntos	Dinero real y puntos virtuales tratados como conceptos separados.	Currencias unifica ambos: los puntos son una moneda más, con código PTS.

Segunda evaluación — 88/100

Mejoras reconocidas

- Implementación del patrón Ledger para todos los movimientos de saldo.
- Normalización completa mediante tablas catálogo.
- Configuración temporal de reglas de negocio (vigencia por fechas).
- Mejor trazabilidad y auditoría de las operaciones del sistema.

Recomendaciones finales

- Implementar System-Versioned Temporal Tables para historiales automáticos.
- Validar el contenido de columnas JSON a nivel de base de datos.
- Utilizar Stored Procedures transaccionales para operaciones críticas.
- Agregar índices optimizados a las tablas de mayor volumen de consultas.

Conclusión

El agente permitió transformar un diseño inicial deficiente en una arquitectura empresarial robusta, auditable y escalable, pasando de 34/100 a 88/100 — una mejora de 54 puntos— en un solo ciclo de revisión.